
Step 1: Preparation of the workspace

Open the regular Windows Explorer.

1. Create a new empty folder, for example - D:\Projects\Calculator\
2. Enter it.
3. Directly in the Explorer's address bar, erase the path, type cmd and press **Enter**.

Result: A black Command Prompt window opens in front of you, which already "looks" into the needed folder.

Step 2: Simple launch

The AutoJunior installer has already added its path to the system variables, you no longer need to search for where the AI is located. You just type the launch command:

```
Microsoft Windows [Version 10.0.19045.xxxx]  
(c) Microsoft Corporation. All rights reserved.
```

```
D:\Projects\Calculator> aid
```

(If you want the AI to first think through the architecture, enter aaid)

Step 3: Aider interface

The program instantly initializes Git (so you can revert unsuccessful changes) and fetches the local Qwen model you downloaded.

```
=====  
Aider v0.xx.x  
Model: ollama/qwen2.5-coder:14b  
Git repo: .git with 0 files  
Repo-map: disabled  
=====
```

```
(Calculator) >
```

Step 4: Communication and task definition

Now you interact with him as if with a living junior who sits at the neighboring desk.

```
(Calculator) > Create a file calculator.py. Write a simple console calculator program in it that can add, subtract, multiply, and divide.
```

Aider analyzes your request, generates code, and independently creates a file in your folder.

```
Thinking... I will create the file `calculator.py` with basic mathematical operations and a simple console interface for the user.
```

```
[Creating file: calculator.py]  
Committing calculator.py to git.
```

```
(Calculator) >
```

Step 5: Testing and refinement

You open your code editor (for example, VS Code) and see the ready file. Suppose you ran it, you liked everything, but you want to add a feature. You return to the terminal:

```
(Calculator) > Great. Now add support for exponentiation.
```

```
Thinking... Got it. I will add the `**` operator for exponentiation and update the menu.
```

```
[Updating file: calculator.py]  
Committing changes to git.
```

□ Useful commands for working inside Aider:

If you are inside the interface (Calculator) >, here are a couple of main hotkeys:

- /add <filename> — add an existing file to the context so the AI can read and modify it.
- /drop <filename> — remove the file from the AI's memory so it won't distract itself with it.
- /undo — if the AI wrote bad code, this command will instantly revert the file to the previous state (thanks to Git).
- /exit (or Ctrl+C) — exit the program.

The whole process boils down to a loop: opened a folder -> wrote aid -> asked to write/edit code -> checked the result. And all of it runs completely locally, without internet or subscriptions!